Istruzioni, blocchi e cicli

Marco Cerbella

Lezione 13 – 26 ottobre 2026

1 Istruzioni

Un'*istruzione* (*statement*) specifica una o più azioni da eseguire, come assegnare un valore a una variabile, passare il controllo a una funzione o saltare a un'altra istruzione. L'insieme di tutte le istruzioni di un programma determina ciò che il programma fa.

I salti e i cicli sono istruzioni che controllano il flusso del programma. Eccetto quando queste istruzioni di controllo producono dei salti, le istruzioni vengono eseguite in sequenza, cioè nell'ordine in cui compaiono nel programma. Un'istruzione semplice è un'espressione seguita da un punto e virgola:

```
[espressione] ;
```

Esempio. `a + 3` è un'*espressione*; `a + 3;` è un *comando* (istruzione). Si veda la differenza:

```c
int a = 3;
a = a + 2;        // istruzione: espressione seguita da ;
if (a + 3;) { }   // SBAGLIATO: dentro le parentesi dell'if serve un'espressione
    , non un'istruzione
```

1.1 Programmazione imperativa e procedurale

In informatica la *programmazione imperativa* è un paradigma di programmazione che usa istruzioni che cambiano lo stato del programma. La *programmazione procedurale* è un paradigma derivato dalla programmazione strutturata, basato sul concetto di chiamata di procedura. Le procedure sono dette anche routine, sottoprogrammi o funzioni. Il C è un linguaggio imperativo e procedurale.

1.2 Altri paradigmi

La *programmazione dichiarativa* è un paradigma che esprime la logica di una computazione senza descriverne il flusso di controllo: descrivendo *che cosa* il programma deve ottenere, piuttosto che *come* ottenerlo, si cerca di minimizzare o eliminare gli effetti collaterali. Un esempio in Prolog:

```
mother_child(trude, sally).
father_child(tom, sally).
father_child(tom, erica).
father_child(mike, tom).
sibling(X, Y) :- parent_child(Z, X), parent_child(Z, Y).
parent_child(X, Y) :- father_child(X, Y).
parent_child(X, Y) :- mother_child(X, Y).
```

```
8
9  ?- sibling(sally, erica).
10 Yes
```

2 Istruzione composta (blocco)

Un'*istruzione composta*, detta in breve *blocco*, raggruppa una serie di istruzioni e dichiarazioni tra parentesi graffe in modo da formare un'unica istruzione:

`{ [lista di dichiarazioni e istruzioni] }`

A differenza delle istruzioni semplici, i blocchi *non* sono terminati da un punto e virgola.

Un blocco si usa dove la sintassi richiede una sola istruzione, ma lo scopo del programma ne richiede più di una.

```
1 {
2     int a = 5, b = 4, c = 0;
3     c = a + b;
4 }
```

Esempio. Senza le graffe l'`if` controlla soltanto la prima istruzione che lo segue:

```
1 int a = -4;
2 if (a < 0)
3     printf("%d is less than 0", a);
4     a = -a;
5     printf("a now is %d", a);
```

```
-4 is less than 0 a now is 4
```

Con `int a = 4;` nello stesso codice `a = -a;` e la seconda `printf` vengono comunque eseguite, perché non sono sotto il controllo dell'`if`; l'output è `a now is -4`. Per controllare tutte e tre le istruzioni occorre un blocco:

```
1 int a = 4;
2 if (a < 0) {
3     printf("%d is less than 0", a);
4     a = -a;
5     printf("a now is %d", a);
6 }
```

2.1 Blocchi e scope

```
{   double result = 0.0, x = 0.0;
    long status = 0;
    int limit;
    ++x;                    // istruzioni
    if (status == 0)
    {                       // nuovo blocco
        int i = 0;
        { int k = 3; }      // un altro blocco
    }
}
```

I nomi dichiarati all'interno di un blocco hanno *block scope*: sono visibili solo dalla loro dichiarazione fino alla fine del blocco. All'interno di questo scope, una dichiarazione può anche nascondere un oggetto con lo stesso nome dichiarato fuori dal blocco.

3 Istruzioni di definizione

Servono a definire variabili. La definizione di una variabile equivale alla sua creazione in memoria. Nello stesso scope le variabili devono avere nomi diversi.

```
int main() {
    int a = 4;      // corretto
}
```

```
int main() {
    int a = 4;
    float a = 5;    // errore: a è già definita in questo scope
}
```

Non si possono usare come espressioni:

```
int main() {
    int a = int b = 3;    // errore
}
```

4 Cicli

Si usa un *ciclo* per eseguire un gruppo di istruzioni, detto *corpo del ciclo*, più di una volta. In C un ciclo si introduce con una delle tre istruzioni di iterazione:

1. `while`;
2. `do ... while`;
3. `for`.

Il numero di iterazioni del corpo è controllato da una condizione, l'*espressione di controllo*: un'espressione di tipo scalare, cioè un'espressione aritmetica o un puntatore. La condizione del ciclo è vera se il valore dell'espressione di controllo è diverso da 0, altrimenti è considerata falsa.

4.1 Istruzione while

Un'istruzione `while` esegue un'istruzione ripetutamente finché l'espressione di controllo è vera:

```
while ( espressione ) istruzione
```

Il `while` è un ciclo *a controllo in testa* (*top-driven*): prima si valuta la condizione del ciclo (l'espressione di controllo). Se è vera, si esegue il corpo e si valuta di nuovo l'espressione di controllo. Se la condizione è falsa, l'esecuzione prosegue con l'istruzione che segue il corpo del ciclo.

Sintatticamente il corpo del ciclo è *una* istruzione; se ne servono più d'una si raggruppano in un blocco.

Esempio.

```c
#include <stdio.h>
int main() {
    int a = 10;

    while (a < 20) {
        printf("value of a: %d\n", a);
        a++;
    }
    return 0;
}
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

4.2 Istruzione for

Come il `while`, l'istruzione `for` è un ciclo a controllo in testa, ma con più logica del ciclo contenuta nell'istruzione stessa:

```
for ( [espressione1]; [espressione2]; [espressione3] ) istruzione
```

- `espressione1`: *inizializzazione*. Viene valutata una sola volta, prima della prima valutazione dell'espressione di controllo, per eseguire le inizializzazioni necessarie.
- `espressione2`: *espressione di controllo*. Viene verificata prima di ogni iterazione; l'esecuzione del ciclo termina quando vale falso.

- `espressione3`: *aggiustamento*, per esempio l'incremento di un contatore, eseguito dopo ogni iterazione del ciclo e prima che l'espressione di controllo venga verificata di nuovo.

Esempio.

```
int a = 0;
int i;
for (i = 0; i < 4; ++i) {
    a = a + 2;
    printf("%d", a);
}
```

```
2468
```

La variabile `i` vale successivamente $0, 1, 2, 3$ (e 4, quando il ciclo termina). Lo stesso comportamento si ottiene con `for (i = 1; i <= 4; ++i)`.

Ognuna delle tre espressioni nell'intestazione del `for` può essere omessa; la forma più breve possibile è quindi `for (;;)`. Un'espressione di controllo mancante è considerata sempre vera e definisce quindi un ciclo infinito. `for (; espressione; )` equivale a `while (espressione)`. Ogni `for` si può riscrivere come `while` e viceversa:

```
int a = 0, i = 0;
while (i < 4)
{
    a = a + 2;
    ++i;
}
```

Dichiarazione al posto di espressione1

Dallo standard C99 si può usare una dichiarazione al posto di `espressione1`. In tal caso lo scope della variabile dichiarata è limitato al ciclo `for`:

```
for (int i = 0; i < 4; ++i) {
    a = a + 2;
}
```

La variabile `i` dichiarata in questo ciclo non esiste più dopo la fine del ciclo.

4.3 Istruzione do ... while

L'istruzione `do ... while` è un ciclo *a controllo in coda* (*bottom-driven*):

```
do istruzione while ( espressione );
```

Il corpo del ciclo viene eseguito una volta prima che l'espressione di controllo sia valutata per la prima volta. A differenza di `while` e `for`, il `do ... while` garantisce che venga eseguita *almeno una* iterazione del corpo. Se l'espressione di controllo è vera segue un'altra iterazione; se è falsa il ciclo termina.

Esempio.

```c
#include <stdio.h>
int main()
{
    double number, sum = 0;

    do   // il corpo viene eseguito almeno una volta
    {
        printf("Enter a number: ");
        scanf("%lf", &number);
        sum += number;
    }
    while (number != 0.0);

    printf("Sum = %.2lf", sum);
    return 0;
}
```

```
Enter a number: 1.5
Enter a number: 2.4
Enter a number: -3.4
Enter a number: 4.2
Enter a number: 0
Sum = 4.70
```

5 Riferimenti

Sul libro: scope o campo di azione, sezione 5.13; sezioni 3.4–3.10, 4.1–4.8 (cicli e istruzioni); sezione 15.9 (da poi scordare immediatamente).